

CS342 -OPERATING SYSTEMS

PROJECT 3 REPORT

Project Members:

Mehmet Efe Mutlu

Ahmet Eren Gökalp

Emir Said Bakan

22303326

22302136

22302852

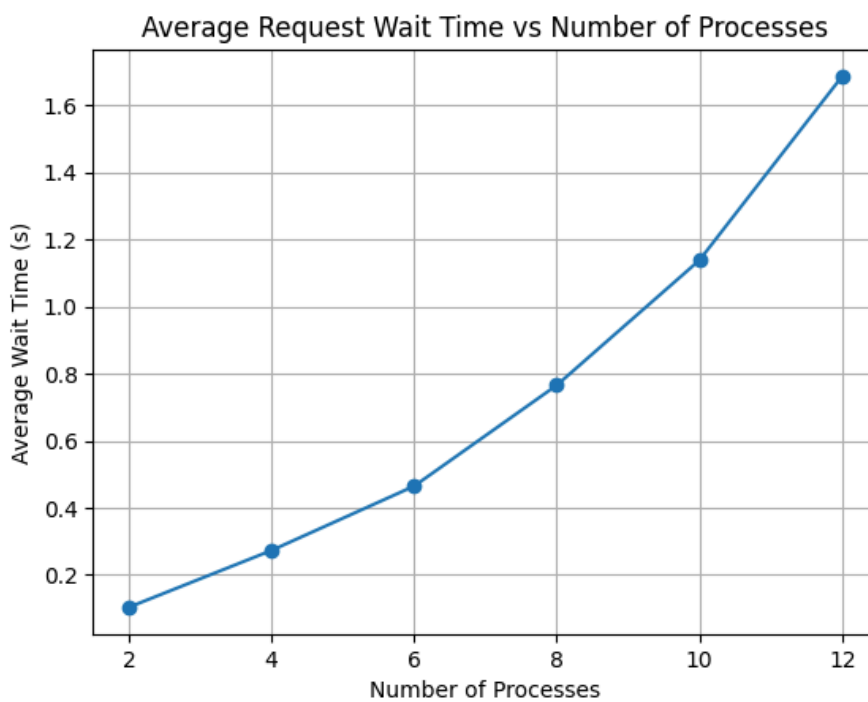
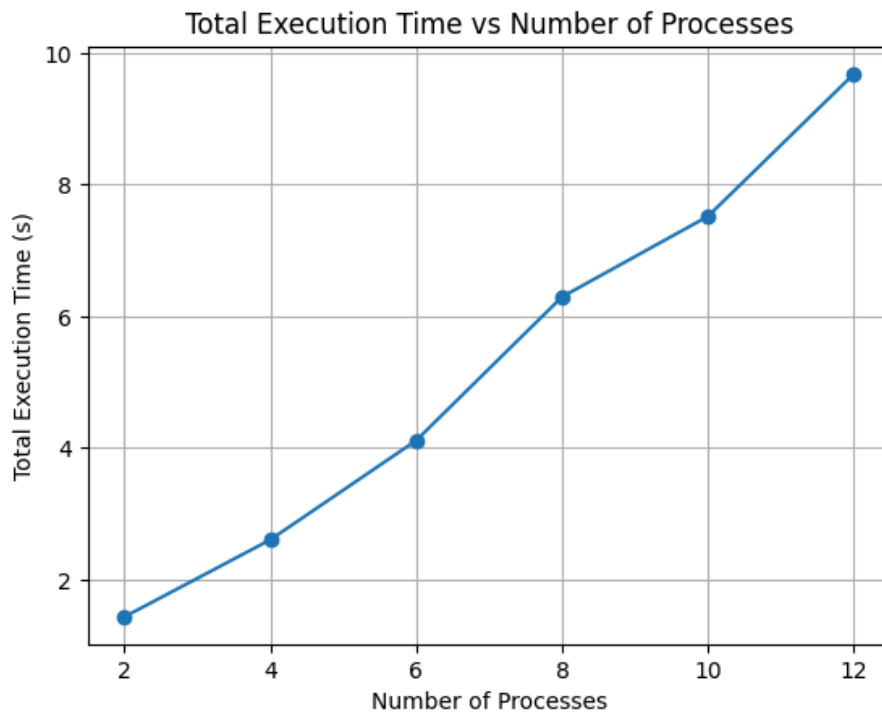
Introduction:

In this project we implemented a Resource Management (RSM) library for a multi-process environment in C on Linux. The library simulates a system in which multiple child processes compete for a fixed number of shared resource types, while the library keeps track of global resource state using POSIX shared memory. Since the processes run concurrently, synchronization is handled with POSIX semaphores to ensure that shared data structures remain consistent and race-free.

Experimental Setup:

All experiments were conducted on a Linux-based system using the *gcc* compiler, consistent with the project environment. The Resource Management Library (RSM) and the demo application were executed under the same system conditions for all trials. In each experiment, one parameter such as the number of processes, the number of resource instances, or the request pattern was selected as the independent variable, while the remaining parameters were kept fixed. The dependent variables were the measured total execution time, average waiting time for resource requests, number of blocked requests, and, when deadlock avoidance was disabled, the number of deadlocked processes detected. The reported values reflect the observed behavior of the library under each configuration.

Experiment 1): Number of Processes

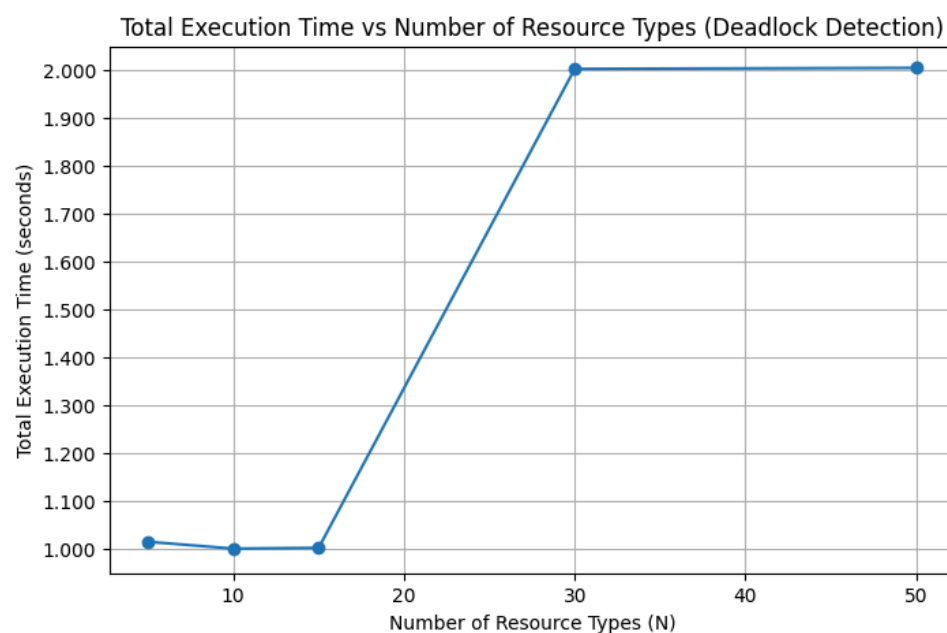


In the number-of-processes experiment, both total execution time and average request wait time increase as the number of child processes grows. For smaller process counts such as 2 or 4, resource contention is relatively low, so requests are usually satisfied quickly and the

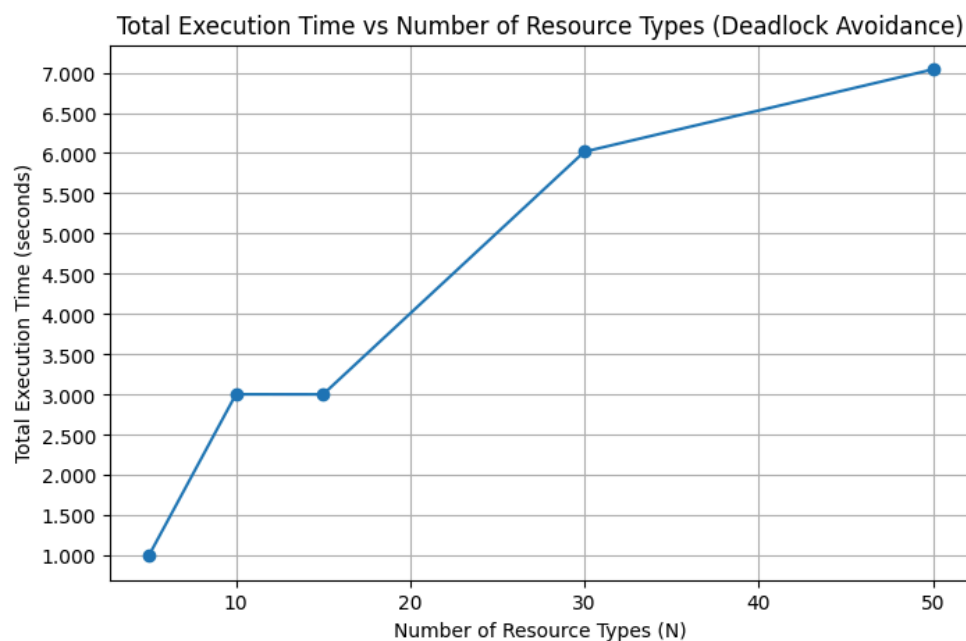
program finishes in a shorter time. As the number of processes increases, more processes compete for the same limited resources, which leads to longer waiting times before requests can be granted. This increase in waiting time directly contributes to the increase in total execution time. The trend shows that the library continues to function correctly as concurrency increases, but the cost of synchronization and resource contention becomes more visible at higher process counts.

Experiment 2): Number of Resource Instances

- **Deadlock Detection (0):**



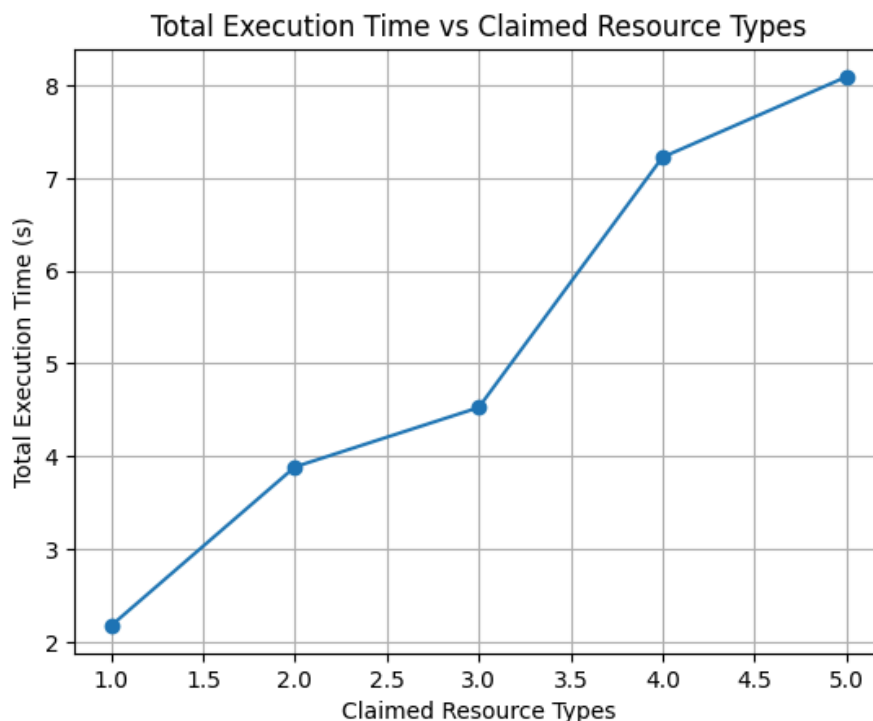
- **Deadlock Avoidance (1):**

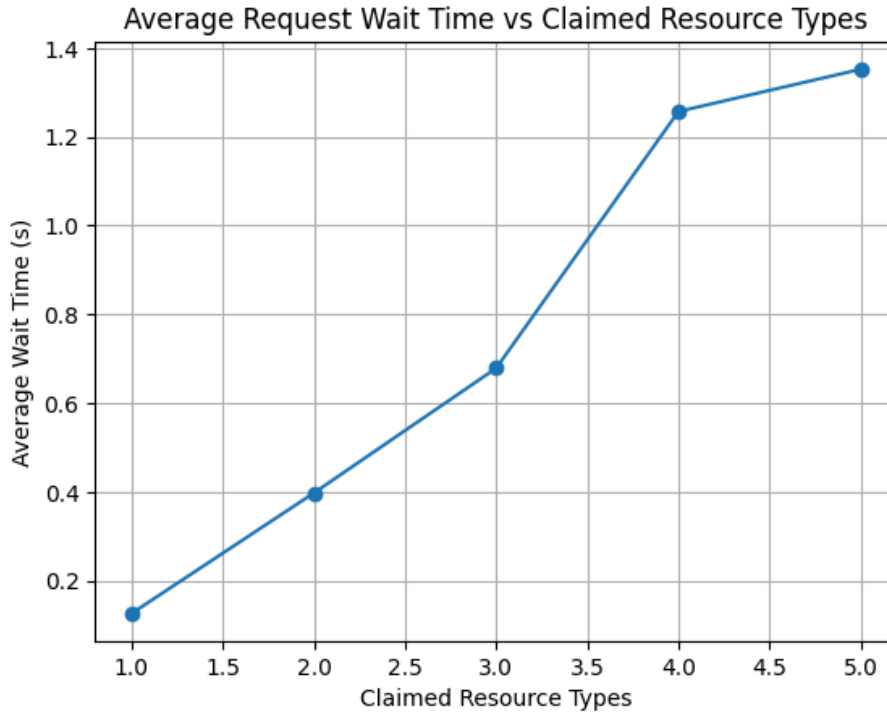


The experimental results show that total execution time increases more noticeably in the deadlock avoidance case as the number of resource types grows. When avoidance is enabled, the execution time rises from about 1 seconds to about 7 seconds. It was expected because the experiment was designed so that, as the number of resource types increases, each process also claims and requests more resources (code has been modified for this purpose). As a result, the system performs more work, and the avoidance mechanism must evaluate more request states to ensure safety before granting resources.

In contrast, the deadlock detection case remains relatively unchanged, with execution times only varying between 1-2 seconds. This suggests that detection mode does not reflect the full growth of workload as clearly as avoidance mode, since execution often stops soon after a deadlock is detected and the processes are evicted before completing all requests. Overall, the results indicate that deadlock avoidance introduces a higher and more visible overhead as the number of resource types increases, while deadlock detection appears less sensitive in this experiment because runs may end early once a deadlock occurs.

Experiment 3): Claimed Resource Types per Process





For this experiment, the number of resource types in the system was kept fixed at five, while the maximum claim vectors provided by the processes were modified to vary the number of claimed resource types per process from 1 to 5. No structural change was made to the library itself; the experiment was carried out by changing the `claim[]` arrays used by the test application before calling `rsm_claim()`. In each configuration, the same number of processes, the same resource instance counts, and the same overall request sequence were preserved so that only the size of the maximum claims acted as the independent variable. Total execution time and average request wait time were then recorded for each run.

The results show that both total execution time and average request wait time increased as the number of claimed resource types per process became larger. When processes declared smaller claims, the system was able to grant requests more quickly, since fewer potential future allocations had to be considered during avoidance checks. As the claim size increased, the library became more conservative in granting requests, because larger maximum demands reduced the number of states considered safe. This caused processes to spend more time waiting before their requests were satisfied, which in turn increased the total execution time. Overall, the experiment suggests that larger claim vectors lead to higher contention and longer execution under deadlock avoidance.